VIDEO PIPELINE OPTIMIZATION AND SCALING (M3)

TEAM: Bhanu Prakash Vangala, Nolan Rink

GITHUB REPOSITORY: https://github.com/bhanuprakashvangala/VideoPipeline

This report documents the implementation of Milestone 3 for the VideoPipeline project. Milestone 3 extends the end-to-end object detection + tracking system with an automated Optimizer that selects pipeline configurations (model variant, batch size, and number of replicas) based on observed load and runtime metrics, while enforcing latency/throughput SLAs and maintaining pipeline accuracy.

1. PIPELINE OVERVIEW AND OBJECTIVE

The core pipeline remains a two-stage architecture: (1) an object detector processes each frame and outputs person bounding boxes with confidence scores, and (2) an OC-SORT tracker associates detections over time to maintain consistent track IDs across frames. Milestone 3 adds an auto-configuration loop (the Optimizer) that adapts pipeline settings to workload fluctuations.

Milestone 3 objectives (mapped to the project tasks):

T1: Define a single pipeline accuracy metric that combines accuracy of multiple stages.

T2: State the optimization problem (choose model/batch/replicas under SLA, cost, and accuracy constraints).

T3 (bonus): Implement and demonstrate a solver (rule-based heuristics and an ILP formulation).

T4 (bonus): Integrate an agentic (Observe–Think–Act) framework using a real LLM to propose configuration actions.

2. DATASET AND MODELS

Dataset: MOT17-04-DPM from the Multi-Object Tracking (MOT17) benchmark.

Frames: 1,050 sequential 1920×1080 images (urban street scene with pedestrians).

Target class: Person (COCO class 0 in YOLO models).

Detection models (variants): YOLOv8n / YOLOv8s / YOLOv8m / YOLOv8l.

Tracking: OC-SORT (Observation-Centric SORT), using IoU and motion modeling to maintain identities.

Hardware: Experiments were run with GPU acceleration when available (example run detected CUDA on NVIDIA L4 with 22 GB).

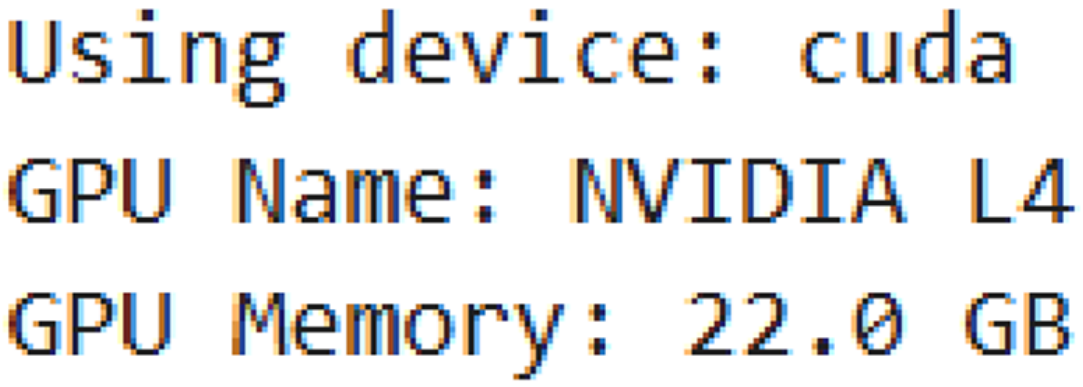

Figure 1: GPU runtime environment detected for experiments (CUDA device, NVIDIA L4).

Detection Models Implemented (Milestone 2):
- YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l (containerized variants)

Tracking Algorithm:
- OC-SORT (Observation-Centric SORT)
- Detection threshold: 0.3
- IoU threshold: 0.3
- Distance threshold: 100 pixels

3. METRICS, CONFIGURATION SPACE, AND SLA

The Optimizer uses monitoring signals collected during execution to make configuration decisions:

• Load: derived from detection volume (e.g., detections-per-frame proxy) and/or request rate.

• Latency: end-to-end processing time per frame (or per request).

• Throughput: frames-per-second (FPS) or requests-per-second (RPS).

• Accuracy: a single combined pipeline accuracy score (defined in Section 4).

• Cost: proportional to allocated resources (replicas × per-replica resource allocation).

• Assertions: runtime consistency checks that detect tracking/detection anomalies (Section 4).

Configuration knobs explored in Milestone 3:

• Model variant: {yolov8n, yolov8s, yolov8m, yolov8l}

• Batch size: {1, 2, 4, 8}

• Replicas: {1, 2, 4, 10} (used to represent horizontal scaling)

Service Level Agreement (SLA) targets used by the Optimizer:

Constraint	Target
Max latency	500 ms
Min throughput	20 FPS
Min accuracy	0.75
Max cost	20

A lightweight profile table is maintained for fast decisions (base latency, cost, and expected accuracy per variant).

Variant	Base latency (ms)	Cost / replica	Expected accuracy
yolov8n	170	1.0	0.861 (measured)
yolov8s	400	2.0	0.850 (assumed)
yolov8m	900	3.0	0.860 (assumed)
yolov8l	1700	4.0	0.870 (assumed)

4. PIPELINE ACCURACY AND ASSERTIONS

Accuracy Definition:

Pipeline accuracy is defined as a weighted combination of three components: detection quality, tracking quality, and temporal consistency. Each component is computed from runtime signals, then aggregated as:

Combined Accuracy = 0.4 × DetectionAccuracy + 0.4 × TrackingAccuracy + 0.2 × ConsistencyAccuracy

Metric	Value
Detection accuracy	0.7120
Tracking accuracy	1.0000
Consistency accuracy	0.8795
Combined accuracy	0.8607

Runtime Assertions:

To detect unstable or implausible behavior during inference and tracking, we implemented six assertions. These are evaluated across frames and tracks and logged as pass/fail counts.

Distance Assertion: Same track ID must not move > 100 pixels between consecutive frames.

Bounding Box Size Assertion: Bounding box area must not change by > 50% between consecutive frames for the same track.

Confidence Threshold Assertion: All reported detections must have confidence ≥ 0.3.

Track Continuity Assertion: Track gaps must not exceed 5 frames (no long disappear/reappear without reason).

Detection Stability Assertion: Detection count per frame must not vary by > 50% between consecutive frames.

IoU Consistency Assertion: Same track's boxes should have IoU ≥ 0.3 between consecutive frames.

Assertion	Passed/Total	Failed	Pass rate
Distance	1191/1191	0	100.00%
BBox size	1189/1191	2	99.83%
Confidence	1221/1221	0	100.00%
Continuity	3/3	0	100.00%
Stability	49/49	0	100.00%
IoU	1191/1191	0	100.00%

Distance Assertion (Track Motion Smoothness)

- Rule: Objects with the same track ID must not move > 100 pixels between consecutive frames
- What it checks: Movement distance of tracked objects between consecutive frames
- Threshold: 100 pixels maximum displacement.
- Why it matters: Objects in video move at physically plausible speeds; a ~100px jump in 1 frame (at ~30fps) suggests a tracking error or ID switch
- Failure indicates: Track ID swap, detection flickering, or the tracker losing and re-acquiring the object

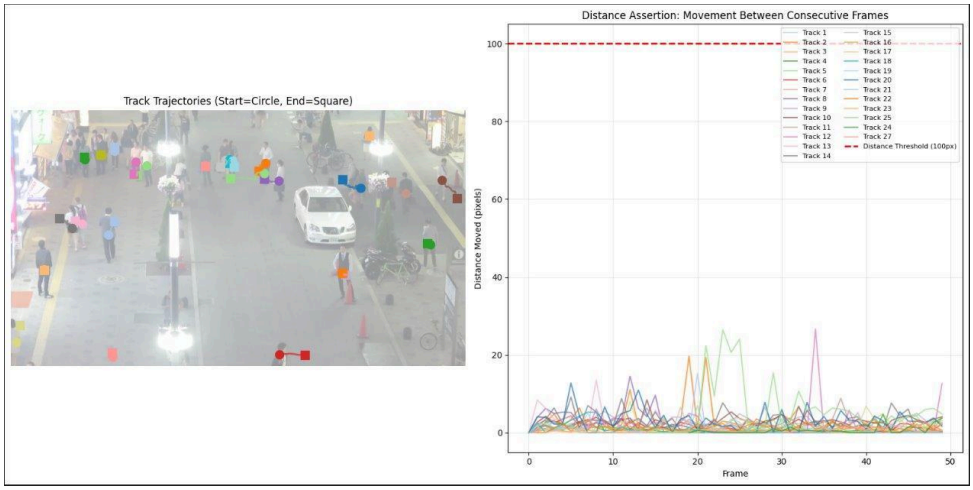

Figure 2: Distance Assertion visualization (track motion vs 100-pixel threshold).

Bounding Box Size Assertion (Scale Stability)

- Rule: Bounding box area shouldn't change > 50% between consecutive frames
- What it checks: Relative change in bounding box area between frames
- Threshold: 50% maximum size change
- Why it matters: Real objects don't suddenly grow or shrink; smooth size changes indicate consistent detection
- Failure indicates: Partial occlusion, detection switching between objects, or detector instability

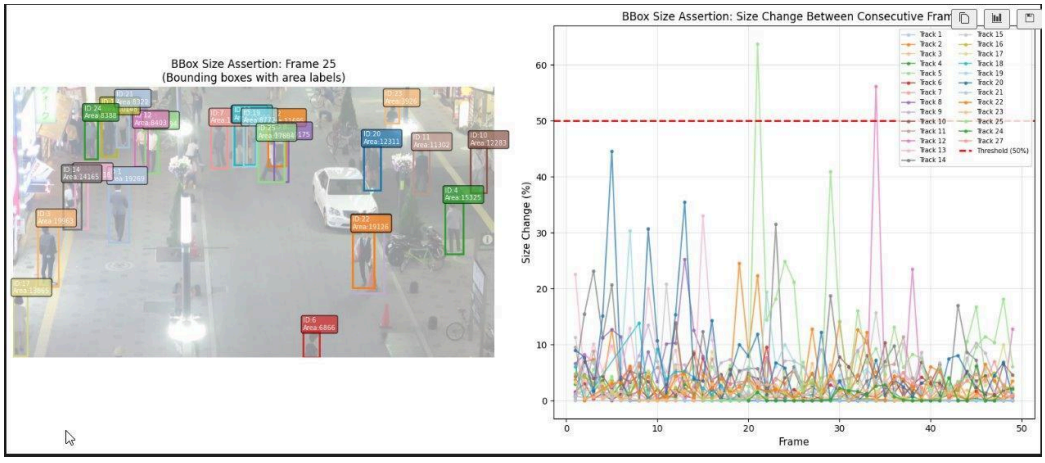

Figure 3: Bounding Box Size Assertion visualization (area change vs 50% threshold).

Confidence Threshold Assertion

1. What it checks: Detector confidence score for each tracked object
2. Threshold: 0.3 minimum confidence

3. Why it matters: Low-confidence detections are unreliable and may be false positives
4. Failure indicates: Detector uncertainty, possible false positive, or an occluded object

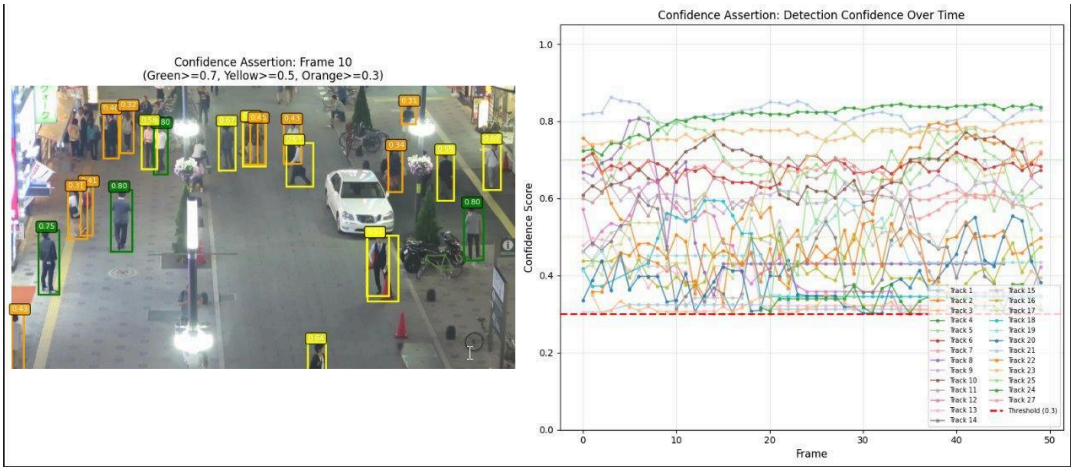

Figure 4: Confidence Assertion visualization (detections vs 0.3 threshold).

Track Continuity Assertion (Gap Constraint)

- What it checks: Frame gaps in track appearance (track disappears then reappears)
- Threshold: 5 frames maximum gap
- Why it matters: Real objects don't teleport; brief gaps can be acceptable (occlusion), but long gaps suggest tracking failure
- Failure indicates: Track fragmentation, prolonged occlusion, or object leaving and re-entering the scene

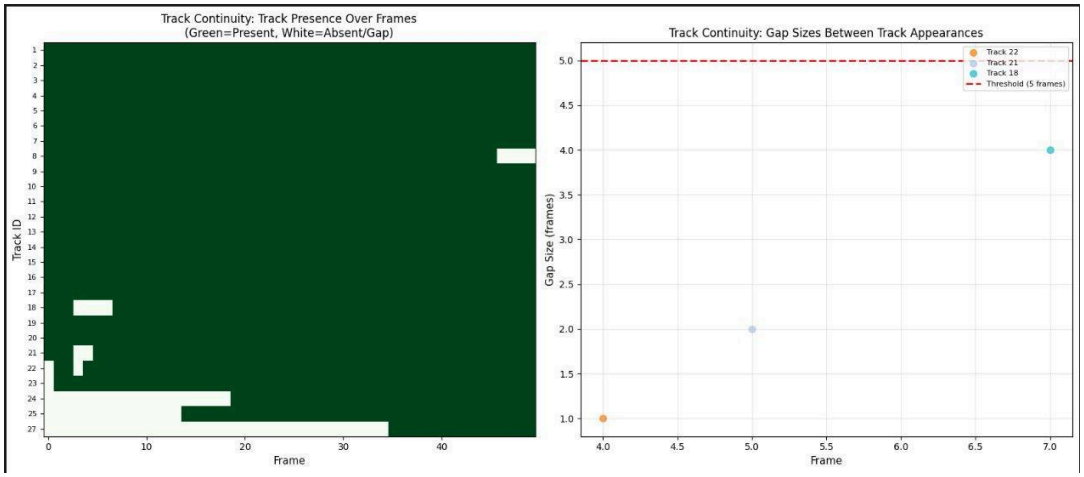

Figure 5: Track Continuity visualization (track presence heatmap and gap sizes vs 5-frame threshold).

Detection Stability Assertion (Count Volatility)

- What it checks: Change in total number of detections between frames
- Threshold: 50% maximum variation
- Why it matters: Scene content typically changes gradually; sudden detection-count changes suggest detector instability
- Failure indicates: Lighting changes, camera motion, detector flickering, or a scene transition

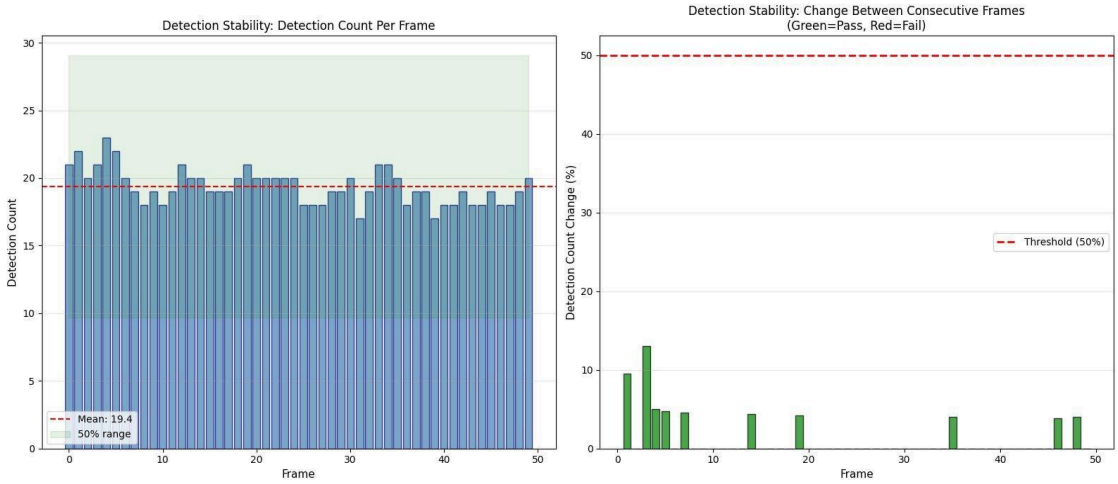

Figure 6: Detection Stability visualization (detection counts and percent change vs 50% threshold).

IoU Consistency Assertion (Box Overlap for Same ID)

- What it checks: Intersection over Union (IoU) between the same track's boxes in consecutive frames
- Threshold: 0.3 minimum IoU
- Why it matters: Consecutive detections of the same object should overlap significantly
- Failure indicates: Rapid object motion, tracking drift, or detection jitter

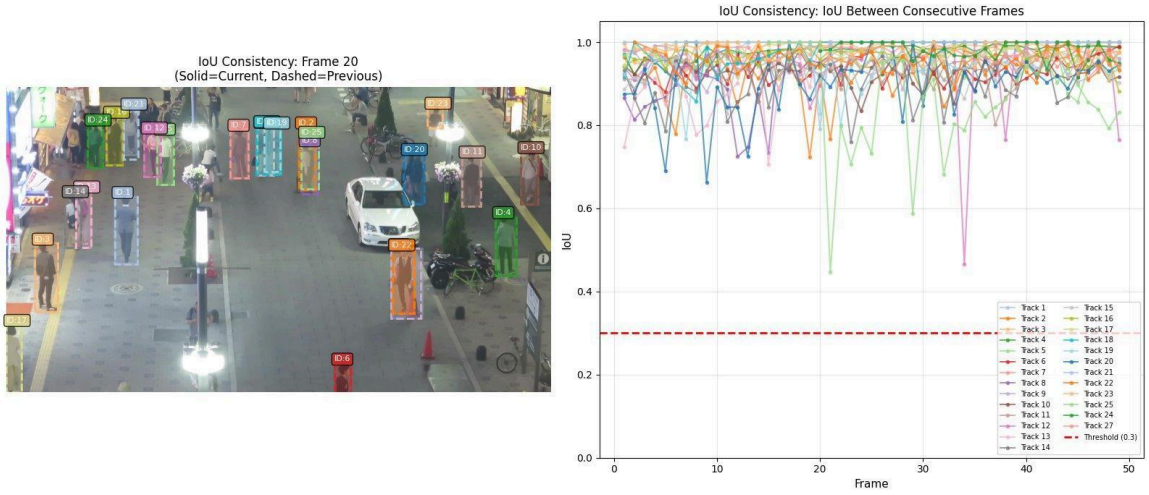

Figure 7: IoU Consistency visualization (IoU across frames vs 0.3 threshold).Note: The only assertion violations observed in this run were 2 bounding-box size changes exceeding the 50% threshold, which can occur during partial occlusions or when detections shift between adjacent objects.

5. OPTIMIZATION PROBLEM AND SOLUTION

Optimization Problem Statement:

At each decision interval, choose a configuration c = (variant v, batch size b, replicas r) that minimizes resource cost while satisfying SLA constraints on latency and throughput and maintaining sufficient accuracy.

Connection to INFaaS (Romero et al., USENIX ATC 2021): Our optimizer is motivated by INFaaS's idea of navigating a space of model variants and adapting both the chosen variant (model-vertical scaling: upgrade/downgrade) and the number of replicas (model-horizontal scaling) as load changes. INFaaS shows that developers can specify high-level latency/accuracy requirements while the system selects and switches variants automatically; in our pipeline we similarly choose (variant, batch size, replicas) configurations that satisfy the SLA constraints while minimizing cost. Unlike INFaaS, our prototype uses an offline profile table and simple heuristics (and an optional ILP/solver) over a small, discrete configuration set, and applies changes via Docker Compose scaling as a stand-in for Kubernetes-based actuation.

Reference: Romero et al., "INFaaS: Automated model-less inference serving," USENIX ATC 2021.

Objective (example): minimize Cost(v, r) subject to:

• Latency(v, b, r, load) ≤ 500 ms

• Throughput(v, b, r, load) ≥ 20 FPS

• Accuracy(v) ≥ 0.75

• Cost(v, r) ≤ 20

Solver Implementation:

We implemented both (1) a rule-based heuristic policy and (2) an ILP formulation using a profile table (variant latency/cost/accuracy) to enable quick configuration selection. The ILP uses a binary decision variable for each candidate configuration and enforces the SLA constraints; the objective minimizes cost (or a weighted score).

Observed Optimization Results (rule-based selections):

Scenario	Variant	Batch	Replicas	Latency (ms)	Throughput (FPS)	Cost
Low Load	yolov8n	8	1	323.00	37.65	1.0
Normal	yolov8n	8	1	323.00	37.65	1.0
High Load	yolov8n	8	10	102.14	376.47	10.0

All selected configurations met the SLA constraints (≤500 ms latency and ≥20 FPS throughput) while maintaining the measured combined accuracy of 0.8607.

6. AGENTIC OPTIMIZER WITH LLM

We integrated an Observe, Think, Act agent to propose configuration actions during runtime. The agent observes metrics (detections, tracks, latency, load proxy, confidence),

thinks by querying a LLM, and acts by selecting one of four actions: MAINTAIN, SCALE_UP, SCALE_DOWN, or SWITCH_MODEL.

Implementation details:

• The agent calls the LLM periodically and reuses the last response between calls to reduce overhead.

• LLM backend: NRP LLM API, using a strict response format to enable parsing.

• Outputs: LLM responses are logged to JSON, and a demo video is produced with a side dashboard summarizing the agent's state.

In the recorded run, the system logged 5 LLM calls with an average LLM response latency of 2714 ms, and executed 1 agent decision(s) based on the responses.

7. DELIVERABLES (Milestone 3)

• Jupyter Notebook: milestone3_complete.ipynb (accuracy metric, assertions, and optimizer experiments).

• Results: milestone3_complete_results.json (T1 accuracy, assertion pass rates, and T3 optimization selections).

• Agentic demo: t4_LLM_based_analysis.py and t4_llm_responses.json (LLM decisions and timing).

• Docker configuration: Dockerfile (+ docker-compose.yml) for reproducible execution.

8. DOCKER ENVIRONMENT

A containerized environment is provided to ensure reproducibility across machines. The Dockerfile pins the PyTorch + CUDA stack and installs required dependencies (Ultralytics YOLO, OpenCV, NumPy, etc.). The docker-compose.yml exposes Jupyter (port 8888), mounts notebooks/results for persistence, and enables GPU use when available.

Typical usage: docker compose build, then docker compose up, then open Jupyter at http://localhost:8888 to run milestone3_complete.ipynb.

9. CONCLUSIONS

Milestone 3 turns the pipeline into an adaptive system by (1) defining a combined pipeline accuracy metric, (2) implementing runtime assertions for detection/tracking consistency, and (3) adding an Optimizer that selects configurations under latency/throughput/accuracy/cost constraints. The bonus agentic framework demonstrates how an external LLM can be incorporated to recommend configuration actions, complementing profile-driven heuristics.